USER MANAGEMENT AND FILE PERMISSIONS

MANAGING USERS

USER ACCOUNTS

- Form the basis of Linux system security and access control
- A user is a member of at least 1 group, and may be a member of multiple groups
- Associated system files
  - /etc/passwd – contains the list of user accounts
  - /etc/shadow – contains the hashed passwords of user accounts

THE PASSWD FILE

root:x:0:0:root:/root:/bin/bash

- Default shell
- Home folder
- Comment / Notes
- Primary GID
- UID
- Password placeholder
- Username

SYSTEM ACCOUNTS

- Are accounts for system services and are necessary for the system to function correctly.
- Usually have a UID of 0 to 499
- Examples
  - Root - account that has access to all commands and files
- Should not be used for login and should not be deleted (unless you know exactly what you're doing)

LOGGING IN AS ROOT

- As much as possible, avoid logging in directly using the root account
  - You might run commands that accidentally cause issues to the system
  - You might accidentally leave the account logged in
- Notes: on some distributions, the root account is disabled by default

THE SHADOW FILE

sysadmin:$6….rl1:15020:5:30:7:60:15050:

- Account expiration
- Password inactive
- Warning period
- Max password validity
- Min password age before change
- Last password change
- Hashed password

GROUP ACCOUNTS

- Also determine the access of a user
- A user is always a member of 1 primary group, and is optionally a member of 1 or more secondary (supplementary) groups
- Information in stored in /etc/group file

mail:x:12:mail,postfix

- Member list (secondary)
- GID
- Group name

VIEWING ACCOUNT INFO

whoami

- Shows the username of the current active user

who

- Shows the list of currently logged on users

id [account]

- View information about an account
- Defaults to the current active account if issued without an argument

Option	What it does
-g	Show primary group
-G	Show secondary groups

MANAGING GROUPS

groupadd [name]

- Creates a new group

groupmod [name]

- Modifies group info

Option	What it does
-i [id]	Changes group ID (affects permissions)
-n [name]	Changes group name (does not affect permissions)

groupdel [name]

- Deletes a group

MANAGING USERS

useradd / usermod / userdel [name]

- Creates / modifies / deletes a user account

Option	What it does
-g [GID / name]	Specify primary group membership
-G [GID1, GID2,...]	Specify secondary groups
-d [absolute path]	Specify home directory
-s [shell]	Specify user shell
-c [comment]	Add comment
-r	Delete user home directory (for userdel)
-m	Create home directory (for useradd)

- Default values when creating an account are located in **/etc/default/useradd** and **/etc/login.defs** files

MANAGING USERS

- On Debian-based systems, alternative commands may be available to manage users and automate certain tasks associated with user management (e.g. creating home folders)
  - **adduser [name]** (create user with profile pattered after /etc/skel)
  - **deluser [name]** (does not remove home folder)
  - **addgroup [name]**
  - **delgroup [name]**

MANAGING USERS

passwd [user]

- Allows changing of a user's password
- If used without a username, changes the password of the current user
-

MANAGING USERS

chage [name]

- Manages password aging options for a user

Option	What it does
-l	List info
-E [expire date]	Set expiration date of account
-I [days]	Set grace period in days
-m [days]	Set minimum password age before allowing change
-M [days]	Set maximum password age
-W [days]	Set number of days before max password age for warning

- Default values when creating an account are located in **/etc/default/useradd** and **/etc/login.defs** files

MANAGING FILE AND FOLDER PERMISSIONS

LINUX FILE AND FOLDER PERMISSIONS

- Linux file and folder access is controlled using a set of permissions

owner	group	world
rwx	rwx	rwx

- Note: When a user is the owner of a file or directory, only the owner permissions are checked even if the user is also a member of the group

Permission	For Files	For Directories / Folder
Read	Open file for viewing or copying	List directory contents (names only)
Write	Modify file contents	Create, delete, rename files in the directory (also requires X permission to work properly)
E**X**ecute	Execute file (e.g. scripts and programs)	Use 'cd' command on the directory and use it in a path. View directory metadata and file contents

LINUX FILE AND FOLDER PERMISSIONS

Scenario 1:

Can bob access test.txt?

drwxr-xr-x root root /

drwxr-xr-- root root /data

-rwxr-xr– bob bob /data/test.txt

Answer: No

Bob has no execute permission on the data directory and cannot enter it to access test.txt

LINUX FILE AND FOLDER PERMISSIONS

Scenario 2:

Who can list the content of the data directory?

Answer: Everyone

Only the 'r' permission is needed to list the names of directory content

drwxr-xr-x root root /

drwxr-xr-- root root /data

-rwxr-xr– bob bob /data/test.txt

LINUX FILE AND FOLDER PERMISSIONS

Scenario 3:

Who can delete test.txt?

drwxr-xr-x root root /

drwxr-xr-- root root /data

-rwxr-xr– bob bob /data/test.txt

Answer: Root user only

A ‘w’ permission on the directory is needed to delete a a file

LINUX FILE AND FOLDER PERMISSIONS

Scenario 4:

Can Bob read the contents of test.txt?

Answer: Yes

Only an 'x' permission is needed to navigate through a directory

dr-xr-xr-x root root /

dr-xr-x--x root root /data

-rwxr-xr– bob bob /data/test.txt

LINUX FILE AND FOLDER PERMISSIONS

Scenario 5:

Can Bob read the contents of test.txt?

Answer: No

Bob owns the file, but the file has no permissions for its owner. The group permissions are not checked because bob is the owner of the file

drwxr-xr-x root root /

dr-xr-x--- bob bob /data

----r-xr– bob bob /data/test.txt

ASSIGNING FILE AND FOLDER PERMISSIONS

chgrp [group] [file/folder]

- Changes the group ownership of a file or folder (-r to include subitems)
- If executed without root privileges, the file or folder ownership may be reassigned only to a group that the user also belongs to

chown [user] [file/folder]

- Changes the user ownership of a file or folder
- May be executed only with root privileges to change user ownership

ASSIGNING FILE AND FOLDER PERMISSIONS

chmod [permission] [file/directory]

- Setting permissions

Field	Choices
Target	**u** (owner) **g** (group) **o** (world) **a** (all)
Operation	**+** (add) **–** (remove) **=** (set to)
Permissions	**r** (read) **w** (write) **x** (execute) **t** (sticky)

Example: chmod g+w file1.txt chmod a=r-x script.py

- Alternative is to use numeric method where each set of permissions is treated as a 3-bit value expressed in decimal form:

Example: chmod 755 script.py = rwxr-xr-x permissions

THE STICKY BIT

- The sticky bit permission prevents non-root, non-owners from deleting a file or directory
- When set on a directory, users may add or delete files, but cannot delete files that they do not own
- When displayed in a listing, items with the sticky bit will have permissions that look like this:

owner	group	world

rwxrwxrwt

't' for sticky

ACCESS CONTROL LISTS

- ACLs provide a more flexible manner of granting access to files and directories
- Permissions can be set on a per user / per group basis rather than the generalized permissions that the traditional method is capable of
- To enable ACL capability, a file system needs to be mounted with the ACL option
  - Temporary mounting

    mount -t ext3 -o **acl** /dev/sda1 /data
  - Edit the /etc/fstab file to make the setting permanent

    /dev/mapper/NETOPSY-root / ext4 errors=remount-ro,**acl** 0 1

SETTING ACCESS CONTROL LISTS

setfacl [option] [entry] [file/directory]- Configures access list entry

Option	What it does
-m	Modify entry
-x	Delete entry

- Access lists can be set

	Syntax	Sample
Per user	**u**:uid:perm	setfacl –m u:alice:rwx test.txt
Per group	**g**:gid:perm	setfacl –m g:student:rx test.txt
Effective Mask	**m**:perm	setfacl –m m:rx test.txt
Users outside the group	**o**:perm	setfacl –m o:rwx test.txt
As a default on a directory	**d**: [u/g/m] perm	setfacl –m d:u:rx dir1

VIEWING ACCESS CONTROL LISTS

getfacl

- Shows the access list of a file or directory
- Sample output

```
# file: abc
# owner: someone
# group: someone
user::rw-
user:johny:rwx
group::r-
mask::rwx
other::r-
default:user::rwx
```

- Owner permission
- Additional user permission
- Group permission
- Effective permission (applies to all except owner)
- World permission
- Default user permission